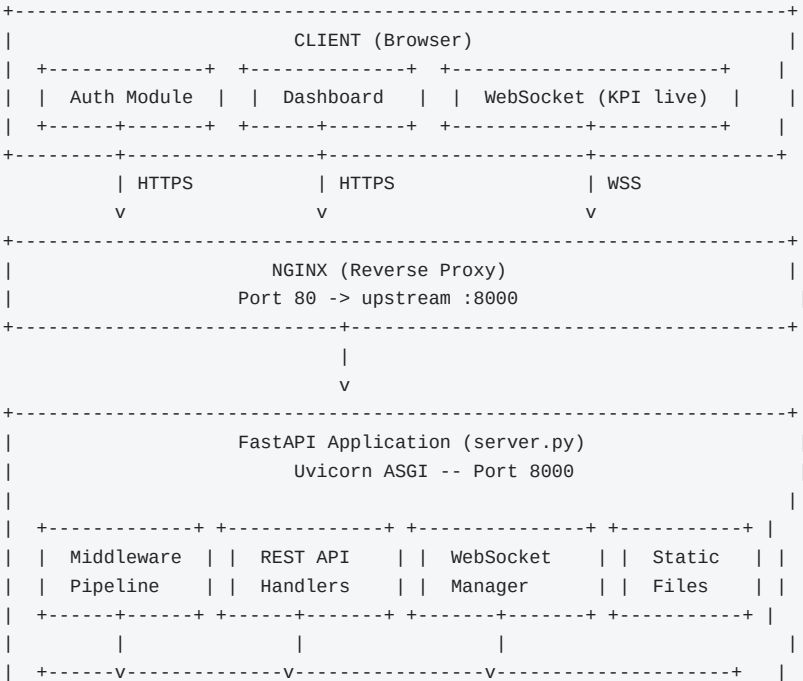


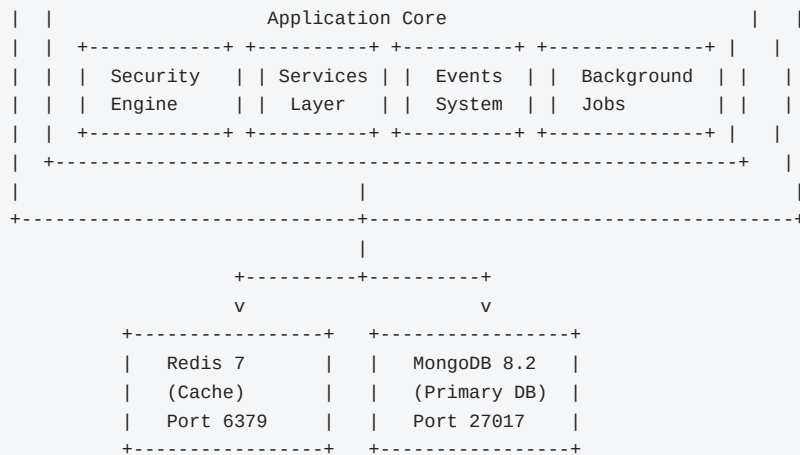
Architecture Document -- IBMS (Integrated Business Management Suite)

Field	Detail
Project Name	Integrated Business Management Suite (IBMS) v2.0
Team	Shivansh Srivastava (Product Developer), Prahallad Padhan (Product Owner), Ranveer Rai Khare (Scrum Master)
Sprint	Sprint 1
Date	April 12, 2026
Document Type	System Architecture

1. System Overview

IBMS follows a **monolithic backend + SPA frontend** architecture served from a single FastAPI process. The backend handles REST APIs, WebSocket connections, background scheduling, and static file serving. **Mongo DB 8.2** is the primary persistence layer (accessed via Motor async + PyMongo sync drivers). Redis is used as an optional caching layer with an automatic in-memory fallback.





2. Layer Architecture

2.1 Presentation Layer (Frontend)

```

frontend/
+-- index.html          <- Single-page application entry
+-- tailwind.config.js  <- Tailwind CSS configuration
+-- static/
    +-- css/
        | +-- dashboard.css <- Custom dashboard styles
        | +-- tailwind-entry.css
    +-- images/          <- Static assets (logos, etc.)
    +-- js/
        +-- app.js       <- IBMS JavaScript engine (SPA router, auth, API, WebSocket)
  
```

- * **SPA Architecture:** Single HTML page with JavaScript-driven page routing
- * **IBMS Namespace:** All frontend logic encapsulated in a global **IBMS** object
- * **Modules:** **auth**, **api**, **ws** (WebSocket), **dashboard**, **toast**, **sidebar**
- * **Chart.js:** Used for forecast, analytics, and KPI history charts
- * **No build step:** Vanilla JS served directly (no bundler required)

2.2 API Layer (server.py)

The server is a single FastAPI application with the following middleware stack executed on every request:

```
Request -> Rate Limiter -> Security Headers -> CORS -> Trace ID -> Route Handler -> Response
```

Middleware Pipeline (execution order):

Order	Middleware	Purpose
1	CORS Middleware	Allow configured origins, credentials, headers
2	Custom Security MW	Rate limiting (IP + user), security headers (CSP, X-Frame-Options, etc.), trace ID injection, response timing

Global Response Headers Applied:

Header	Value
`X-Content-Type-Options`	`nosniff`
`X-Frame-Options`	`DENY`
`X-XSS-Protection`	`1; mode=block`
`Referrer-Policy`	`strict-origin-when-cross-origin`
`Permissions-Policy`	`camera=(), microphone=()`
`Content-Security-Policy`	`default-src 'self'; ...`
`X-Trace-Id`	UUID per request
`X-Response-Time-Ms`	Elapsed milliseconds

2.3 Application Core Layer

```

apps/ibms_core/ibms_core/
+-- __init__.py
+-- auto_budget_optimizer.py      <- Budget allocation with growth targets
+-- compliance_engine.py         <- Transaction control-set validation
+-- digital_twin.py              <- Operational simulation engine
+-- risk_scoring_engine.py       <- Composite risk scoring (weighted factors)
|
+-- security/                    <- Authentication & Authorization
|   +-- auth_engine.py           <- JWT, 2FA, RBAC, password policy, device binding
|   +-- jwt_auth.py              <- JWT encode/decode (HS256)
|   +-- zero_trust.py            <- Service identity validation
|   +-- audit_logger.py          <- Compliance audit trail
|   +-- oauth_provider.py        <- OAuth configuration
|   +-- policies.py              <- Access control policies
|   +-- auth_hooks.py            <- Frappe auth lifecycle hooks
|
+-- services/                    <- Business Intelligence Services
|   +-- ai_assistant.py          <- NL query engine for business metrics
|   +-- anomaly.py               <- Outlier detection (transactions/metrics)
|   +-- behavioral_analytics.py  <- User behavior profiling

```

```

| +-- decision_engine.py      <- Workflow approval routing
| +-- dynamic_pricing.py     <- AI pricing (demand/stock/competitors)
| +-- fraud_detection.py     <- Isolation Forest fraud scoring
| +-- kpi_engine.py          <- KPI aggregation & calculation
| +-- lead_scoring.py        <- Lead qualification (engagement + fit)
| +-- predictive_inventory.py <- Inventory forecasting
|
+-- events/                  <- Publish-Subscribe Event System
| +-- event_router.py        <- Topic-based event routing
| +-- publisher.py           <- Event publishing (invoice submit, webhook)
| +-- stream_processor.py    <- Batch event & webhook processing
| +-- subscriber.py          <- Event handler routing
|
+-- jobs/                    <- Background Task Scheduler
| +-- kpi_rollup.py          <- Aggregate KPI snapshots
| +-- compliance_check.py    <- Detect stale webhooks, generate recommendations
| +-- process_webhook_queue.py <- Poll & process webhook logs
| +-- auto_workflow_optimizer.py <- Approval chain optimization proposals
| +-- retrain_models.py      <- ML model retraining (ml-heavy queue)
| +-- nightly_kpi_refresh.py <- Nightly KPI recalculation
|
+-- monitoring/              <- Observability
| +-- healthcheck.py         <- DB connectivity probe
| +-- metrics.py             <- Request counters & metric snapshots
| +-- tracing.py             <- UUID trace ID generation
|
+-- api/                     <- Additional API modules
| +-- auth.py, rest.py, risk.py, forecast.py, dashboard.py
| +-- analytics.py, ai_assistant.py, ai_copilot.py
| +-- integrations.py, graphql_api.py, graphql_schema.py
|
+-- doctype/                 <- Frappe DocType definitions
| +-- ai_recommendation/     <- AI-generated suggestions
| +-- enterprise_profile/    <- Company profiles
| +-- integration_webhook_log/ <- Webhook event log
| +-- kpi_snapshot/          <- KPI data snapshots
| +-- smart_decision_rule/    <- Business rule definitions

```

2.4 Data Layer

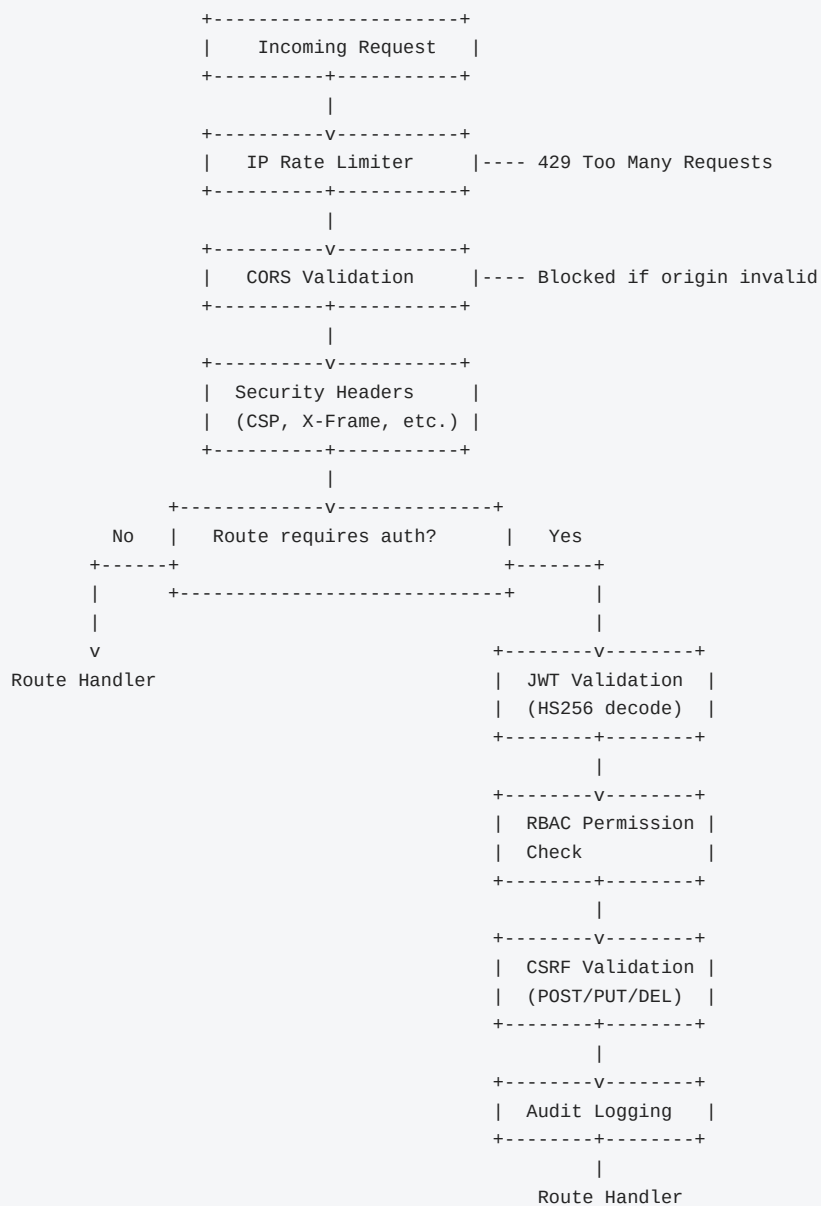
+-----+-----+-----+	+-----+-----+-----+
Redis 7	MongoDB 8.2
(Cache & Session Store)	(Motor async + PyMongo)
- KPI snapshot cache	13 Collections:
- Rate limit counters	- users (unique email/username)
- Session tokens	- kpi_snapshots (time-series)
- In-memory fallback if	- kpi_latest (materialized)
Redis unavailable	- audit_logs (timestamped)
	- refresh_tokens (TTL index)
Config:	- csrf_tokens (1hr TTL)
- maxmemory: 256MB	- rate_limits
- eviction: allkeys-lru	- notifications
- appendonly: yes	- alerts, ai_recommendations
	- enterprise_profiles
+-----+-----+-----+	- webhook_logs, decision_rules

```

| Config: |
| - URI: localhost:27017 |
| - DB: ibms_enterprise |
| - TTL indexes on tokens |
+-----+

```

3. Security Architecture



Authentication Flow:

1. **Login** -> Validate credentials -> Check 2FA -> Generate device fingerprint -> Issue access token (30 min) + refresh token (7 days, HTTP-only cookie)
2. **Authenticated Request** -> Extract Bearer token -> Decode JWT (HS256) -> Verify expiry -> Resolve RBAC permissions -> Execute handler
3. **Token Refresh** -> Validate refresh cookie -> Rotate token (old revoked, new issued) -> Device binding check

Password Security:

- * Hashing: bcrypt (12 rounds) with PBKDF2 (SHA256, 310K iterations) fallback
- * Policy: Min 8 chars, mixed case, numeric, special chars, common word blacklist

RBAC Hierarchy:

```

super_admin  ---> Full Access (*)
|
admin        ---> dashboard, users, reports, settings, api, ai, risk, compliance, audit
|
manager      ---> reports.export, risk.manage, compliance.manage, budget.approve
|                (inherits from analyst)
analyst      ---> reports.view, ai.view, risk.view, compliance.view, forecast.view
|                (inherits from viewer)
viewer       ---> dashboard.view, kpi.view

```

4. Deployment Architecture

4.1 Docker Compose (Development / Staging)

```

+-----+
|                                     |
|               Docker Compose Network               |
|                                     |
| +-----+ +-----+ +-----+ +-----+ |
| | Nginx | |-->| Web (API) |-->| Redis | | MongoDB | | |
| | :80   | | | :8000   | | | :6379 | | :27017 | |
| | alpine | | | python:3.11 | | | 7-alpine | | mongo:7 | |
| +-----+ +-----+ +-----+ +-----+ |
|                                     | Volumes: | Volume: |
|                                     | redis_data | mongo_data |
+-----+

```

4.2 Kubernetes (Production)

```

+-----+
|               Kubernetes Cluster               |
+-----+

```

```

|
| +-----+
| | Ingress Controller |
| | Host: ibms.example.com -> Service:ibms-core:8000 |
| | TLS termination, /api path routing |
| +-----+
| |
| +-----v-----+
| | Service: ibms-core (ClusterIP :8000) |
| +-----+
| |
| +-----v-----+ +-----v-----+ +-----v-----+
| | Pod: ibms-1 | | Pod: ibms-2 | | Pod: ibms-3 |
| | CPU: 500m-2 | | CPU: 500m-2 | | CPU: 500m-2 |
| | Mem: 1Gi-4Gi | | Mem: 1Gi-4Gi | | Mem: 1Gi-4Gi |
| | Probes: [x] | | Probes: [x] | | Probes: [x] |
| +-----+ +-----+ +-----+
|
| Probes:
| - Readiness: /api/method/ping (delay 15s, period 10s)
| - Liveness: /api/method/ping (delay 45s, period 20s)
| +-----+

```

4.3 AWS Infrastructure (Terraform)

```

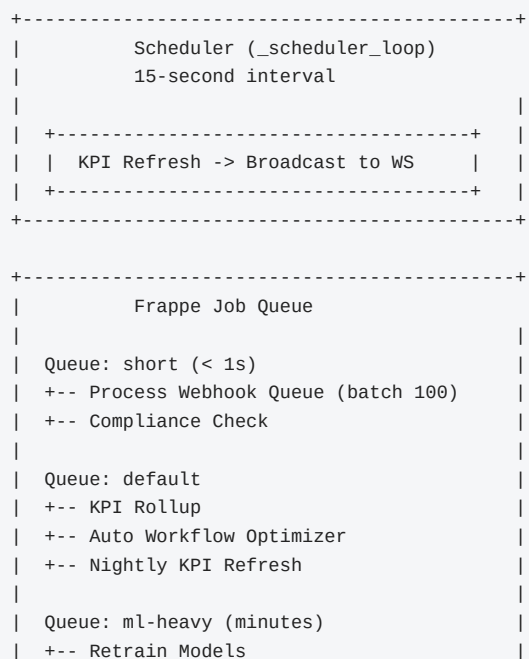
+-----+
|
|           AWS VPC (10.70.0.0/16)
|
| +-----+ +-----+
| | Private Subnet AZ-1 | | Private Subnet AZ-2 | | | | |
| | 10.70.1.0/24 | | 10.70.2.0/24 |
| | | |
| | +-----+ | +-----+ |
| | | MongoDB 8.2 | |<-----?| | Replica Set | |
| | | (Atlas / | | Replica | | (Failover) | |
| | | DocumentDB) | | | |
| | +-----+ | | +-----+ |
| | | | | |
| | +-----+ | | +-----+ |
| | | ElastiCache | |<-----?| | Redis Replica |
| | | Redis | | Auto | | (Failover) |
| | | t3.medium | | Failover | |
| | +-----+ | | +-----+ |
| +-----+ +-----+
|
| Note: MongoDB runs locally for dev; production can use
| MongoDB Atlas or DocumentDB in private subnets.
|
| Security:
| - App SG: Port 8000 (internal only)
| - MongoDB: Private subnet, auth enabled
| - Redis: At-rest + transit encryption, private subnet
| - Backups: 7-day retention (MongoDB Atlas / AWS Backup)
| +-----+

```

5. Event-Driven Architecture



6. Background Job Architecture



+-----+

7. Data Flow -- Key User Journey

7.1 User Login -> Dashboard View

Browser	API Server	Redis	DB
--POST /api/auth/login-->			
{user, pass, device}	--verify password---->		
	--check 2FA----->		
	--issue JWT-----		
?--{access_token, csrf}--			
+ Set-Cookie: refresh			
--GET /api/dashboard---->			
Authorization: Bearer	--cache_get("kpi")---->		
	?--{kpi_data}-----		
?--{revenue, margin...}--			
--WS /ws/kpi?token=JWT-->			
?=== kpi_update push ===	(every 15s)		

8. Technology Decisions

Decision	Rationale
FastAPI over Flask/Django	Async support, auto OpenAPI docs, Pydantic validation, high performance
Monolith over Microservices	Simpler deployment for v1, all modules share same process
Redis with in-memory fallback	Graceful degradation; project runs without Redis dependency
JWT over Session cookies	Stateless auth, suitable for API-first architecture
Vanilla JS over React/Vue	No build toolchain needed; fast iteration for dashboard
Terraform for AWS	Reproducible infrastructure, version-controlled, multi-AZ
Docker Compose for dev	One-command local environment setup
K8s for production	Auto-scaling, self-healing, rolling deployments

Document Version 1.0 -- Sprint 1 Review